

SISTEMA DE REPORTES FARMAPATRIA

Guía de Arquitectura, Tecnologías y Cuestionario de Estudio para Defensa de Tesis

GUÍA DE TECNOLOGÍAS Y CUESTIONARIO DE ESTUDIO

Sistema de Reportes e Incidencias Técnicas — Farmapatria (Proyecto de Tesis)

Resumen Ejecutivo del Proyecto

El **Sistema de Reportes Farmapatria** es una plataforma web integral diseñada para la gestión, seguimiento, diagnóstico y resolución oportuna de incidencias técnicas y fallas de servicios en la red nacional de farmacias y dependencias de Farmapatria.

El sistema fue concebido bajo estándares modernos de ingeniería de software:

- **Arquitectura Cliente-Servidor Desacoplada** basada en una API RESTful.
 - **Frontend SPA (Single Page Application)** rápido, reactivo y moderno.
 - **Backend Modular por Capas** con división estricta de rutas, controladores, servicios y middlewares.
 - **Seguridad Multicapa:** Sesiones sin estado con JWT, encriptación con BCrypt, protección de rutas con Guards e interceptores en cliente, middlewares en servidor y autenticación de dos factores (2FA / MFA).
-

1. Arquitectura General del Sistema

El sistema opera bajo un esquema desacoplado donde la interfaz de usuario y la lógica de datos funcionan como entidades independientes comunicadas a través del protocolo HTTP/HTTPS:

1. **Capa Cliente (Frontend):** Angular 21 + Tailwind CSS v4 + TypeScript + Chart.js + jsPDF.
2. **Capa de Transporte y Acceso:** Cloudflare Tunnels (acceso remoto seguro cifrado) + CORS.
3. **Capa Servidor (Backend):** Node.js + Express.js v5 con arquitectura por capas (MVC).
4. **Capa de Persistencia:** Base de datos relacional PostgreSQL con pool de conexiones gestionado por `pg (node-postgres)`.



2. Inventario y Ficha Técnica de Tecnologías

A. Frontend (Presentación y Experiencia de Usuario)

1. Angular (v21.2)

- **Qué es:** Framework de desarrollo web para aplicaciones de una sola página (SPA).
- **Función en el Sistema:** Administra toda la experiencia del usuario en el navegador sin recargas de página, implementando la arquitectura moderna de *Standalone Components*.

- **Concepto Clave para la Tesis:** Garantiza modularidad, renderizado dinámico y desacoplamiento total de la interfaz frente a la base de datos.

2. TypeScript (v5.9)

- **Qué es:** Superconjunto tipado de JavaScript desarrollado por Microsoft.
- **Función en el Sistema:** Lenguaje en el que está desarrollado el frontend. Define modelos e interfaces de dominio (`Ticket` , `Usuario` , `Servicio`).
- **Concepto Clave para la Tesis:** Detección temprana de errores en tiempo de compilación y robustez en la integración con la API.

3. Tailwind CSS (v4.1)

- **Qué es:** Framework de diseño CSS utilitario de última generación.
- **Función en el Sistema:** Estilizado completo y responsivo de paneles, tablas interactivas, formularios de incidencias y modales.
- **Concepto Clave para la Tesis:** Permite construir interfaces modernas, adaptables a teléfonos y computadoras con máxima velocidad de carga.

4. Angular Router y Guards de Navegación

- **Qué es:** Sistema de rutas de Angular con mecanismos de protección.
- **Función en el Sistema:** Gobierna las rutas de la aplicación y protege vistas críticas mediante:
 - `AuthGuard` : Bloquea el acceso a usuarios no identificados.
 - `AdminGuard` / `RoleGuard` : Bloquea el panel administrativo a usuarios sin roles técnicos o de superadministrador.
- **Concepto Clave para la Tesis:** Primera barrera de seguridad perimetral a nivel de interfaz de usuario.

5. Interceptor HTTP (`authInterceptor`)

- **Qué es:** Middleware del cliente HTTP de Angular (`provideHttpClient`).
- **Función en el Sistema:** Intercepta automáticamente cada petición saliente hacia la API e inyecta la cabecera `Authorization: Bearer <token>` leyendo el JWT desde el almacenamiento local (`localStorage`).
- **Concepto Clave para la Tesis:** Centralización de la seguridad en el cliente: ningún componente necesita gestionar manualmente las cabeceras de autorización.

6. Chart.js (v4.5)

- **Qué es:** Librería de renderizado de gráficos interactivos basada en HTML5 Canvas.
- **Función en el Sistema:** Alimenta los indicadores clave de rendimiento (KPIs) en el panel de administración (volumen de tickets por estado, incidencias por gerencia, etc.).
- **Concepto Clave para la Tesis:** Apoya la toma de decisiones gerenciales mediante tableros visuales analíticos.

7. jsPDF (v4.2)

- **Qué es:** Librería JavaScript para creación y exportación de documentos PDF.
- **Función en el Sistema:** Permite que los técnicos y administradores generen reportes impresos de fallas o resúmenes de casos directamente desde el navegador.

- **Concepto Clave para la Tesis:** Generación distribuida en el cliente sin sobrecargar el procesador del servidor backend.

8. RxJS (v7.8)

- **Qué es:** Librería para programación reactiva con Observables.
- **Función en el Sistema:** Maneja las operaciones asíncronas del cliente, flujos de datos y respuestas HTTP del backend.

B. Backend (Servidor, Lógica de Negocio y APIs)

1. Node.js

- **Qué es:** Entorno de ejecución (runtime) de JavaScript del lado del servidor basado en el motor V8 de Google Chrome.
- **Función en el Sistema:** Plataforma de ejecución orientada a eventos con entrada/salida (I/O) no bloqueante.
- **Concepto Clave para la Tesis:** Gran capacidad para procesar altas peticiones concurrentes de farmacias a nivel nacional con mínimo consumo de memoria.

2. Express.js (v5.2)

- **Qué es:** Framework web minimalista y robusto para Node.js.
- **Función en el Sistema:** Expone todos los servicios RESTful (rutas para tickets, usuarios, autenticación, notificaciones y reportes de servicios).
- **Concepto Clave para la Tesis:** Manejo ágil de enrutamiento y soporte moderno para promesas y cadenas de middlewares.

3. Arquitectura por Capas (Patrón MVC Modular)

- **Estructura Interna:**
 - `src/routes/` : Definición de endpoints y métodos HTTP (GET, POST, PUT, DELETE).
 - `src/controllers/` : Orquestación del flujo, recepción de datos y respuestas JSON.
 - `src/services/` : Lógica de negocio dura y ejecución de sentencias SQL.
 - `src/middlewares/` : Filtros de seguridad, auditoría y carga de archivos.
 - `src/config/` : Conexión a base de datos y variables de entorno.
- **Concepto Clave para la Tesis:** Separación de responsabilidades que garantiza mantenibilidad y código limpio.

4. express-validator (v7.3)

- **Qué es:** Conjunto de middlewares de validación y sanitización para Express.
- **Función en el Sistema:** Valida rigurosamente formatos de entrada (cédula numérica, correos válidos, contraseñas seguras, longitud de descripciones de tickets).
- **Concepto Clave para la Tesis:** Blindaje contra datos corruptos e inyecciones a nivel de entrada de API.

5. Multer (v2.1)

- **Qué es:** Middleware para procesamiento de solicitudes con formato `multipart/form-data`.

- **Función en el Sistema:** Gestiona la carga de fotos de perfil y archivos de evidencia adjuntos a los reportes (capturas de error, facturas o actas técnicas).
- **Medidas de Seguridad:** Filtra tipos MIME (JPG, PNG, GIF, PDF) y restringe el tamaño a un máximo de 5MB por archivo.

6. CORS (Cross-Origin Resource Sharing v2.8)

- **Qué es:** Estándar de seguridad del navegador para peticiones de origen cruzado.
- **Función en el Sistema:** Permite la conexión segura entre el puerto del frontend (4201) y la API del backend (3000).

7. Dotenv (v17.4)

- **Qué es:** Gestor de variables de entorno.
- **Función en el Sistema:** Mantiene ocultas y aisladas las credenciales de PostgreSQL, secretos JWT y puertos en el archivo `.env`.

8. Nodemon (v3.1)

- **Qué es:** Monitor de desarrollo.
- **Función en el Sistema:** Reinicia en caliente el servidor Node.js al detectar cambios en el código.

C. Seguridad, Criptografía y Control de Acceso

1. JSON Web Tokens - JWT (`jsonwebtoken` v9.0)

- **Mecanismo:** Autenticación sin estado (*Stateless*).
- **Función en el Sistema:** Al iniciar sesión, el servidor firma un token digital que contiene el ID y Rol del usuario con caducidad de 8 horas.
- **Concepto Clave para la Tesis:** No ocupa memoria en base de datos para sesiones y permite una verificación criptográfica ultrarrápida.

2. Bcrypt.js (v3.0)

- **Mecanismo:** Algoritmo de hashing adaptativo unidireccional con salting.
- **Función en el Sistema:** Encripta las claves con un salt de 10 rondas antes de almacenarlas. Las contraseñas nunca se guardan en texto plano.
- **Concepto Clave para la Tesis:** Protección estándar de la industria contra ataques de fuerza bruta y tablas arcoíris (*Rainbow Tables*).

3. Autenticación de Dos Factores - 2FA / MFA (`speakeasy` + `qrcode`)

- **Mecanismo:** Algoritmo TOTP (Time-based One-Time Password).
- **Función en el Sistema:** Proporciona un segundo factor de autenticación a usuarios administrativos generando códigos QR compatibles con Google Authenticator.
- **Concepto Clave para la Tesis:** Seguridad avanzada para mitigar robos de identidad y accesos indebidos.

4. Auditoría de Acciones Administrativas (`audit.middleware.js`)

- **Mecanismo:** Trazabilidad y no repudio.
- **Función en el Sistema:** Guarda en la tabla `auditoria_root` cada operación crítica (modificación de roles, bloqueo de cuentas, etc.), registrando usuario, endpoint, detalles en JSONB y fecha/hora.

5. Manejador Global de Errores (`errorHandler.middleware.js`)

- **Mecanismo:** Resiliencia y Principio Fail-Safe.
- **Función en el Sistema:** Atrapa fallos no controlados, evitando caídas del servidor (`crashes`) y previniendo que se filtren datos internos de PostgreSQL al usuario.

D. Base de Datos (Persistencia de Datos)

1. PostgreSQL

- **Qué es:** Sistema Gestor de Bases de Datos Relacional (RDBMS) de clase empresarial.
- **Función en el Sistema:** Garantiza almacenamiento seguro, atomicidad en las transacciones e integridad referencial por claves foráneas.

2. pg (`node-postgres v8.20`)

- **Qué es:** Cliente oficial no bloqueante de PostgreSQL para Node.js.
- **Función en el Sistema:** Administra un **Pool de conexiones**, optimizando tiempos de respuesta al reutilizar conexiones abiertas.

Entidades Principales del Modelo de Datos:

1. `usuarios` : Cédula, correo, nombre, apellido, contraseña encriptada, gerencia, farmacia, rol de permisos, estado (Activo/Inactivo/Bloqueado), avatar y credenciales MFA.
2. `tickets` : Reportes técnicos correlativos (`REP-XXXX`), creador, técnico asignado, estados de atención, ID de AnyDesk, clasificación de falla y evidencias adjuntas.
3. `reportes_servicios` : Reportes de indisponibilidad de suministros críticos (electricidad, internet, Cantv, agua) con tiempos sin servicio y proveedores.
4. `notificaciones` : Mensajería interna automatizada para alertar a los usuarios sobre avances en sus reportes.
5. `auditoria_root` : Bitácora de seguridad de acciones privilegiadas.

E. Infraestructura, Despliegue y Redes

1. Cloudflare Tunnels (`cloudflared`)

- **Qué es:** Servicio perimetral de red cifrada de Cloudflare.
- **Función en el Sistema:** Publica el sistema local a internet mediante un túnel saliente seguro (`trycloudflare.com`) sin abrir puertos en el router ni contratar IP fija.
- **Concepto Clave para la Tesis:** Solución rentable y segura para pruebas de campo en distintas farmacias del país durante la evaluación del proyecto.

2. Scripts de Automatización en Lotes (.bat)

- `iniciar_desarrollo.bat` : Arranca en paralelo el backend y el frontend, abriendo el navegador en `http://localhost:4201`.
 - `iniciar_produccion.bat` : Arranca el servidor y establece de inmediato el túnel público con Cloudflare.
-

3. Flujo del Ciclo de Vida de los Tickets

Para la presentación de la tesis es fundamental explicar las 4 etapas del ciclo de atención:

1. **Pendiente:** El usuario de una farmacia detecta una falla, llena el formulario con clasificación, AnyDesk y evidencia, creándose el ticket correlativo.
 2. **En Progreso:** Un técnico de soporte toma el ticket desde su panel. El sistema notifica al creador que su requerimiento está siendo atendido.
 3. **Sin Confirmar:** El técnico solventa la avería (remota o presencialmente) y marca el ticket como concluido. El sistema solicita al usuario que verifique en su puesto de trabajo.
 4. **Resuelto:** El usuario valida el correcto funcionamiento y pulsa "Confirmar Solución". Si la falla persiste, selecciona "Reportar Error Persistente" y el ticket vuelve automáticamente a estado **En Progreso**.
-

4. Banco de Preguntas y Respuestas para Examen de Tesis

Módulo 1: Arquitectura y Conceptos Generales

1. **Pregunta:** ¿Cuál es la arquitectura general del sistema y por qué se escogió?
 - **Respuesta:** Se implementó una arquitectura Cliente-Servidor desacoplada (Frontend SPA en Angular 21 y Backend API RESTful en Node.js/Express 5). Se escogió porque permite escalabilidad independiente, mejor rendimiento de usuario sin recargas de página completa y facilita que la misma API pueda servir a futuras aplicaciones móviles.
 2. **Pregunta:** ¿Cuál es la ventaja de utilizar una Single Page Application (SPA)?
 - **Respuesta:** En una SPA solo se carga el documento HTML una vez. Todas las interacciones subsiguientes descargan únicamente datos en formato JSON mediante llamadas asíncronas, logrando transiciones instantáneas y menor consumo de ancho de banda.
-

Módulo 2: Frontend (Angular & TypeScript)

3. **Pregunta:** ¿Qué es un Guard en Angular y cuáles están presentes en el proyecto?
 - **Respuesta:** Es una interfaz que condiciona la navegación entre rutas según reglas de negocio. En el sistema existen `AuthGuard` (comprueba que el usuario esté autenticado con token válido) y `AdminGuard` (valida que

el rol del usuario corresponda a personal técnico o administrador).

4. **Pregunta:** ¿Cómo garantiza el frontend que las peticiones a la API estén autenticadas?

- **Respuesta:** A través de un `HttpInterceptor` (`authInterceptor`) registrado en la configuración de Angular. Captura todas las solicitudes salientes e inyecta automáticamente el encabezado `Authorization: Bearer <token>` obtenido del `localStorage` .

5. **Pregunta:** ¿Qué tecnologías se usaron para gráficos y reportes descargables?

- **Respuesta:** Se usó `Chart.js` para los indicadores y gráficos interactivos del panel de control, y `jsPDF` para compilar y descargar reportes directamente en formato PDF desde el navegador del cliente.

Módulo 3: Backend y Seguridad (Node.js & Express)

6. **Pregunta:** ¿Qué patrón de diseño sigue el backend y cuál es su beneficio?

- **Respuesta:** Sigue una arquitectura por capas (MVC modular) separando `routes` (rutas), `controllers` (gestión de solicitudes), `services` (lógica de negocio y SQL) y `middlewares` (seguridad y validaciones). Evita código monolítico, mejora la lectura y facilita pruebas unitarias.

7. **Pregunta:** ¿Cómo se almacenan las contraseñas de forma segura en la base de datos?

- **Respuesta:** Se encriptan mediante la librería `bcryptjs` utilizando un factor de salting de 10 rondas. Nunca se guardan en texto plano, lo que impide que las claves sean descubiertas aún ante accesos indebidos a la base de datos.

8. **Pregunta:** ¿Cómo funciona el mecanismo de autenticación mediante JWT?

- **Respuesta:** Al hacer login, el servidor valida las credenciales y devuelve un token firmado digitalmente con clave secreta (válido por 8 horas). En cada solicitud posterior, el middleware `verificarToken` inspecciona la firma criptográfica y extrae el identificador y rol del usuario sin necesidad de consultar la base de datos.

9. **Pregunta:** ¿Qué es y cómo funciona el 2FA implementado en el sistema?

- **Respuesta:** Es un sistema de autenticación en dos factores basado en TOTP (contraseñas de un solo uso por tiempo). Con `speakeasy` se genera un secreto criptográfico único y con `qrcode` se produce el código para escanear en apps como Google Authenticator.

10. **Pregunta:** ¿Cómo se controla la subida de archivos adjuntos para evitar ataques?

- **Respuesta:** Mediante el middleware `multer` , que valida la extensión y el tipo MIME (admitiendo únicamente imágenes JPG/PNG y PDFs) e impone un límite máximo de tamaño de 5MB por archivo.

Módulo 4: Base de Datos (PostgreSQL)

11. **Pregunta:** ¿Qué ventaja tiene el uso de `Pool` en `node-postgres` frente a crear clientes individuales?

- **Respuesta:** Un `Pool` mantiene abiertas conexiones listas para ser reutilizadas en memoria. Esto evita el costo de negociar el protocolo TCP y autenticar la sesión con PostgreSQL en cada consulta, multiplicando la capacidad de usuarios concurrentes.

12. **Pregunta:** ¿Cuáles son las tablas principales del sistema y cuál es su relación?

- **Respuesta:** Las tablas principales son `usuarios` y `tickets` (relacionadas mediante claves foráneas por `usuario_id` y `tecnico_id`), `reportes_servicios` (para averías de servicios generales), `notificaciones` (alertas asociadas a usuarios) y `auditoria_root` (trazabilidad de administración).

Módulo 5: Despliegue, Infraestructura y Redes

13. **Pregunta:** ¿Qué es Cloudflare Tunnels (`cloudflared`) y por qué fue elegido?

- **Respuesta:** Es un conector seguro que establece un túnel saliente hacia los servidores de Cloudflare. Permite publicar la plataforma en internet con cifrado HTTPS (`trycloudflare.com`) sin tener que abrir puertos en el firewall de la organización, sin configurar NAT ni requerir una dirección IP pública estática.

14. **Pregunta:** ¿Cuál es el procedimiento para encender el sistema en producción o pruebas?

- **Respuesta:** Se ejecuta el archivo automatizado `iniciar_produccion.bat` , el cual levanta el servicio backend en Node.js y simultáneamente inicializa el túnel de Cloudflare, entregando de inmediato el enlace web público para acceder desde cualquier equipo con conexión a internet.